AsyncDataProvider

Wojciech Koperny

April 16, 2024

Contents

1 Static System overview **2**
1.1 System objects 3
1.1.1 Dispatcher 3
1.1.2 DataCache 3
1.1.3 TaskPool 3
1.1.4 Task 4
1.1.5 Database 4

2 Dynamic Behaviour **5**
2.1 Client requests data by ID 5

1 Static System overview

The Asynchronous Data Provider system is a generic service that acts as a proxy to a simple key-value database. Each entry in the database is indexed with unique Id. The backend that will handle such storage is yet to be decided. A provider exposes a C++ interface that can be used by external applications directly in the code. It is provided as a shared or static library.

The main goal of the system is to provide concurrent access to the data for multiple threads, thus maintaining the high responsiveness of the system. Each data request is cached in the system and in case of cache miss, a request is passed on a TaskPool queue. Tasks are consumed in parallel and provide results asynchronously directly to the client. It is assumed that retrieval of data from the database might be occupied with high resource usage and be time consuming.

The fgure 1 shows UseCase for AsyncDataProvider.

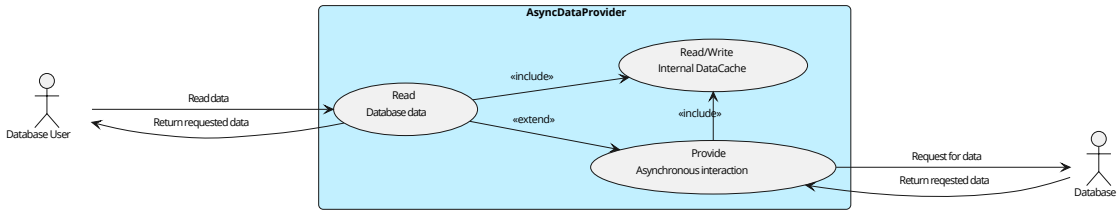


Figure 1: Asynchronous Data Provider Usecase

1.1 System objects

The fgure 2 shows the main system objects.

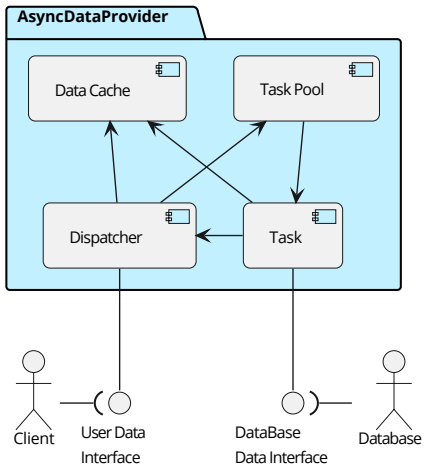


Figure 2: Asynchronous Data Provider system overview

1.1.1 Dispatcher

A Dispatcher is an entity that provides boundary interface for the system receiving the Data Requests form the Client. For every request a `std::future<T>` object is returned.

The Dispatcher frst tries to get the requested data from Cache, if it's available, it sets the `std::promise<T>` immediately. In case the data is not available, Dispatcher creates a new Task and puts in on the TaskPool queue.

1.1.2 DataCache

A Data Cache entity provides Caching functionality for already-requested data. When Data exist in cache (cache "hit") response is imediet. When Data in Cache is not existing (cache "miss") at the end of Task life newly acquired data will be stored into Data Cache for further use.

For every request from Dispacher to DataCache `std::optional<T>` is created and flled with either data or NULL

1.1.3 TaskPool

A TaskPool entity provides queueing for Task functionality. When DataCache "miss" Dispacher calls `TaskPool::enqueTask`. TaskPool shall store task in `std::vector<T>`.

Paralell acces into certain TaskPool resource shall be resurce protected for thread safe (i.e. Mutex mechanism shall be use)

1.1.4 Task

A Task entity is worker object spawned as `std::thread` from TaskPool. It interacts with Database to receive needed Data. Once it receives it sets `std::promise<T>::set_value`, push received data to DataCache for storing it using `DataCache::putData` interface and cease to exist.

Calling `std::promise<T>::set_value` will notify DatabaseClient that data is ready.

1.1.5 Database

Database for testing purpouses can be createn in SQLight

2 Dynamic Behaviour

2.1 Client requests data by ID

Figure 3 shows interaction between Database user, Database and AsyncDataProvider.

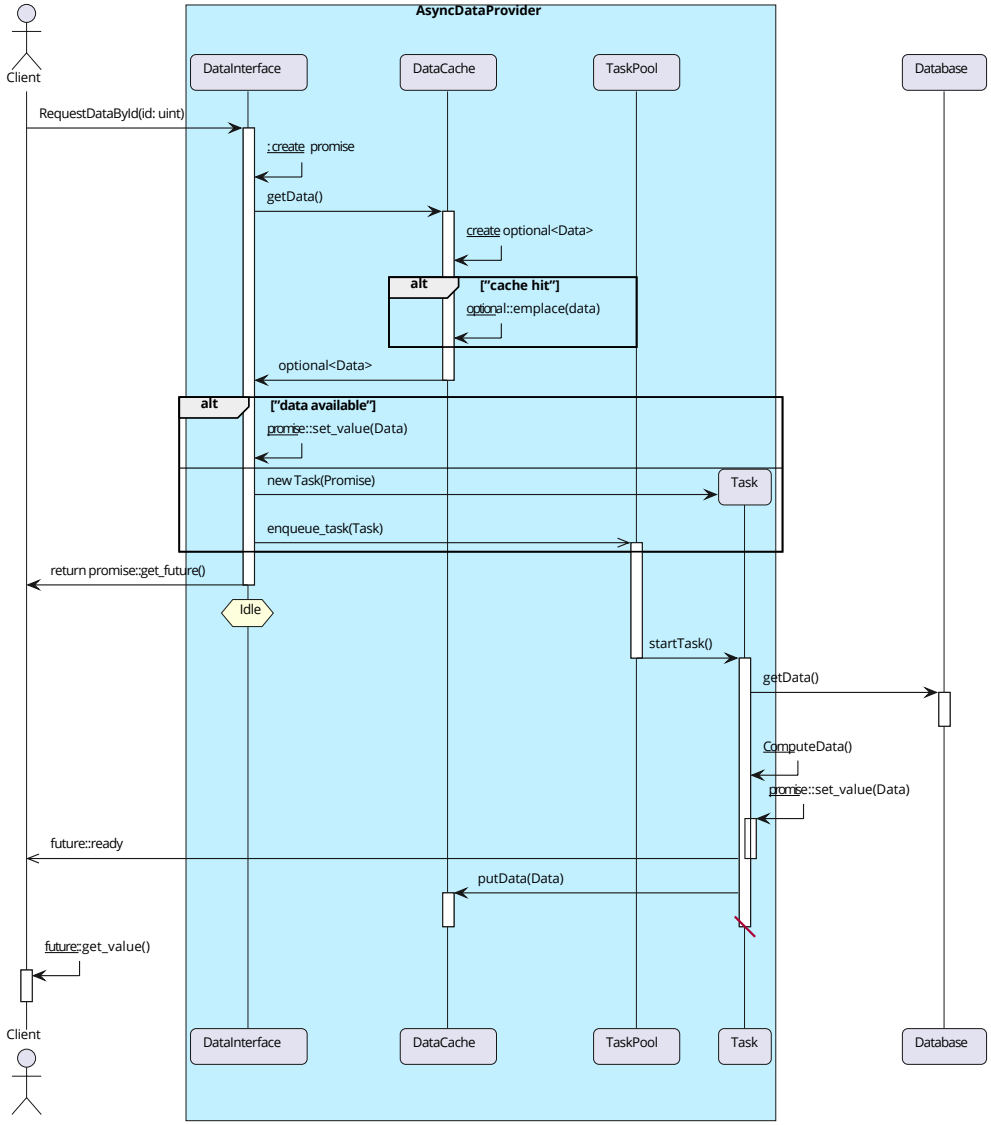


Figure 3: PlantUML diagram from fle